Actividad 4 - Evaluación de riesgo

Juan David Lozano Serna

Docente: John Rodríguez

Universidad Iberoamericana
Calidad de software
2024

Introducción:

La auditoría de ciberseguridad es un proceso indispensable dentro del análisis y la protección de los sistemas de información, porque permite identificar vulnerabilidades, evaluar riesgos y proponer medidas correctivas para así poder fortalecer la postura de seguridad de una organización. En las aplicaciones web, una de las amenazas mas relevantes y comunes es la SQL injection, catalogada por OWASP como una de las vulnerabilidades mas criticas de la industria.

El objetivo principal de este informe es analizar las vulnerabilidades asociadas a la inyección SQL dentro de las aplicaciones web, identificar los riesgos que representa en el panorama actual para la integridad y disponibilidad de la información y finalmente proponer algunos tips o recomendaciones relacionadas a las buenas practicas y recomendaciones basadas en estándares reconocidos como la OWASP Testing Guide y otros marcos de referencia.

Análisis de vulnerabilidades

La inyección SQL ocurre cuando un atacante manipula consultas enviadas a una BD (Base de datos o Database) aprovechando entradas no validadas. Esta falla se origina cuando una aplicación web inserta directamente datos controlados por el usuario dentro de una secuencia SQL sin los controles adecuados.

Estas son las vulnerabilidades mas comunes relacionadas con las inyecciones SQL:

Falta de validación y sanitización de entradas:

Al momento de usarse los formularios en las paginas web y el usuario los digita, estos valores se insertan directamente en las consultas SQL y los atacantes pueden inyectar código malicioso.

Ejemplo:

- `SELECT * FROM usuarios WHERE usuario = '$user' AND clave = '$pass';`

Un atacante podría enviar:

- `' OR '1'='1`

Permitiendo el acceso sin credenciales validas.

Uso de consultas SQL dinámicas
Las consultas ensambladas mediante concatenación de strings son altamente vulnerables. Los atacantes suelen explotarlas para acceder a información sensible o incluso modificar la base de datos

Gestión inadecuada de errores
Los mensajes de error SQL que se muestran al usuario revelan detalles internos como:

- Nombre de tablas
- Estructura de consultas
- Tipos de datos
- Tecnologías del DBMS

Esta información facilita ataques posteriores más precisos.

Falta de controles de autenticación y autorización.
Un ataque de SQL injection puede permitir:

- Omitir login
- Escalar privilegios
- Consultar tablas restringidas
- Insertar o eliminar registros

Configuraciones inseguras a las bases de datos
Muchas de las bases de datos están configuradas con:

- Cuentas con roles excesivos (root o admin)
- Bases de datos sin cifrado
- Falta de auditoría de consultas
- Permisos globales para usuarios de aplicaciones

Esto aumenta el impacto potencial del ataque.

Mejores prácticas y recomendaciones

Las siguientes medidas están alineadas con OWASTP Testing Guide, NIST RMF y buenas prácticas de la industria

Uso obligatorio de consultas preparadas
Evita completamente la concatenación de strings.

Ejemplo

- `SELECT * FROM usuarios WHERE usuario = ? AND clave = ?;`

Validación estricta del lado del servidor

Implementación de listas blancas para

- Longitud permitida
- Formatos validos
- Tipos de datos

Rechazar cualquier input que no cumpla el patrón esperado

Principio de mínimo privilegio

La cuenta con la que la aplicación se conecta a la base de datos debe tener solo permisos necesarios:

- Solo SELECT para operaciones de lectura
- Solo UPDATE/INSERT específicos para tablas puntuales
- Nunca usar root/admin

Gestión adecuada de errores

Mostrar mensajes genéricos al usuario final:

- "Ocurrió un error inesperado, intente más tarde."

Mientras los detalles del error se registran en logs internos del servidor

Uso de un WAF (Web Application Firewall)
Un WAF puede detectar patrones maliciosos como:

- `' OR '1' = '1`
- `UNION SELECT`
- `DROP TABLE`

Reduciendo el riesgo incluso si la aplicación tiene fallas.

Pruebas periódicas de pentesting

Aplicar herramientas como:

- SQLMap
- Burp Suite
- OWASP ZAP

Estas permiten identificar inyecciones presentes antes de que un atacante lo haga.

Cifrado de información sensible

En caso de una filtración, los datos no deben ser legibles:

- Hash seguro para contraseñas (bcrypt, script, Argon2)
- Cifrado AES para campos como dirección o E-mail

Configuración segura del servidor y la base de datos

- Auditar consultas
- Registrar accesos
- Mantener los parches y versiones actualizadas
- Deshabilitar funciones peligrosas (LOAD DATA)

Conclusiones

1. La auditoria de seguridad en aplicaciones web permite identificar de manera objetiva las vulnerabilidades que podrían comprometer la integridad, confidencialidad y disponibilidad de los datos. Comprender como opera una vulnerabilidad como la inyección SQL facilita no solo detectar fallas técnicas, sino también entender los riesgos organizacionales asociados. Este análisis demuestra que incluso un error simple en la validación de entradas puede abrir la puerta a ataques con consecuencias críticas, como la manipulación de información o el acceso no autorizado.
2. Las vulnerabilidades relacionadas con inyecciones SQL siguen siendo una de las principales amenazadas para las aplicaciones web debido a las malas practicas de desarrollo y configuraciones deficientes. La implementación de medidas correctivas basadas en estándares como OWASP y NIST es fundamental para mitigarlas. Aplicar principios como mínimo privilegio, consultas preparadas y validación estricta permite reducir significativamente el riesgo. Estas acciones, además, promueven una cultura de seguridad que trasciende el ámbito técnico y se convierte en un elemento esencial de la gestión empresarial.

3. La importancia de la auditoria en ciberseguridad radica en su capacidad para proporcionar un análisis sistemático y estructurado que orienta la toma de decisiones informada. Al documentar vulnerabilidades y proponer recomendaciones claras, se fortalece la comunicación entre los equipos técnicos y las áreas administrativas. A largo plazo, este proceso mejora la postura de seguridad y reduce la probabilidad de incidentes críticos. Finalmente, este ejercicio evidencia como la inyección SQL no solo representa una amenaza técnica, sino también un riesgo estratégico para cualquier organización que dependa de aplicaciones web.

Referencias Bibliográficas

- Ardila-Osma, J. A., Trujillo-Romero, F., & Marín-Pabón, M. (2020). *Revisión sobre hacking ético y su relación con la inteligencia artificial*. Revista Reto, 8(1), 11–21.
- Bytecode Security. (2023). *Best Way to Learn Ethical Hacking: A Comprehensive Guide*.
- Ciberarrow. (s.f.). *Top 15 pentest tools for ethical hacking. Used by pros*.
- Horcajuelo-Muñoz, J. (s.f.). *Hacking ético: guía completa para hackers éticos*. Sale Systems.
- NIST. (2024). *Risk Management Framework (RMF)*. Computer Security Resource Center.
- OWASP. (2013). *Testing Guide 4.0*.
- OWASP Foundation. (2021). *OWASP Top 10 – 2021*.